Session 6

1. Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.

➔

```
USE db_restaurant;

CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    amount DECIMAL(8,2)
);

INSERT INTO orders (user_id, amount)
VALUES (1, 500), (1, 2150), (2, 1700), (3, 300), (3, 4050), (3, 2000);

SELECT user_id, COUNT(*) AS order_count
FROM orders
GROUP BY user_id;
```

```
sql> • SELECT * FROM `db_restaurant`.`orders`
```

order_id	user_id	amount
1	1	500.00
2	1	2150.00
3	2	1700.00
4	3	300.00
5	3	4050.00
6	3	2000.00

Output | Result #4

user_id	order_count
1	2
2	1
3	3

2. Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.

➔

```
USE db_restaurant;

CREATE TABLE restaurants(
    id INT(11) PRIMARY KEY AUTO_INCREMENT,
    name varchar(255),
    location VARCHAR(255),
    rating DECIMAL(2,1)
);

INSERT INTO restaurants(name, location, rating)
VALUES
('Agashiye', 'Ahmedabad', 4.5),
('Agashiye', 'Ahmedabad', 4.3),
('Leopold Cafe', 'Mumbai', 3.8),
('Leopold Cafe', 'Mumbai', 3.9);

SELECT name, AVG(rating) AS avg_rating
FROM restaurants
GROUP BY name
HAVING AVG(rating) > 4.0;
```

```
sql> SELECT * FROM `db_restaurant`.`restaurants`
```

id	name	location	rating
1	Agashiye	Ahmedabad	4.5
2	Agashiye	Ahmedabad	4.3
3	Leopold Cafe	Mumbai	3.8
4	Leopold Cafe	Mumbai	3.9

Output | Result #4

name	avg_rating
Agashiye	4.40000

3. Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above 2000 in any transaction.

➔

```
USE db_restaurant;

CREATE TABLE payments(
   payment_id INT(11) PRIMARY KEY AUTO_INCREMENT,
   user_id INT(11),
   amount DECIMAL(8,2),
   payment_date DATE
);

INSERT INTO payments(user_id,amount,payment_date)
VALUES
(1, 2500, '2026-11-01'),
(2, 1500, '2026-11-02'),
(3, 800, '2026-11-02'),
(3, 3000, '2026-11-03');

SELECT DISTINCT user_id
FROM payments
WHERE user_id IN(
   SELECT user_id
   FROM payments
   WHERE amount > 2000
);
```

```
sql> SELECT * FROM `db_restaurant`.`payments`
```

payment_id	user_id	amount	payment_date
1	1	2500.00	11/01/2026
2	2	1500.00	11/02/2026
3	3	800.00	11/02/2026
4	3	3000.00	11/03/2026

Output | Result #4

user_id
1
3

4. Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies. Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.

➔

```
USE movie;

CREATE TABLE movies(
    id INT(11) PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255),
    rating DECIMAL(2,1)
);

INSERT INTO movies(name,rating)
VALUES
('Jailer 2',4.8),
('Jailer 2',5.0),
('Pathaan', 4.5),
('Pathaan', 4.0),
('Jawan', 3.5),
('Jawan', 3.0),
('Dunki', 2.5);


SELECT name, AVG(rating) AS avg_rating
FROM movies
GROUP BY name
HAVING AVG(rating) > (
    SELECT AVG(rating)
    FROM movies
);
```

```
sql> SELECT * FROM `movie`.`movies`
```

id	name	rating
1	Jailer 2	4.8
2	Jailer 2	5.0
3	Pathaan	4.5
4	Pathaan	4.0
5	Jawan	3.5
6	Jawan	3.0
7	Dunki	2.5

Output | Result #1

name	avg_rating
Jailer 2	4.90000
Pathaan	4.25000